

Parallel Data Mining

Term Project Report

CP431 Parallel Programming

Instructor: I. Kotsireas

Group 9

2025-03-31

Alexander Lebouthillier, [REDACTED] [REDACTED]

Input processing, frequent item detection, frequent pair generation, cluster testing, code review

Eldan Malenboim, [REDACTED] [REDACTED]

Item name-to-ID conversion, map file generation, rule output formatting, slideshow design,
poster contribution.

Jennifer Sampson, [REDACTED] [REDACTED]

Report structure, report content, project revision, poster

Navid Jussab, [REDACTED] [REDACTED]

Report structure, report content, project revision, poster

Omar Abdeen, [REDACTED] [REDACTED]

Code writing, code testing, report, poster

1. Introduction

1.1 Objective

For this final term project, we set out to mine frequent item-sets and compute association rules using the Apriori data mining algorithm and using parallel programming. Our program can be used for datasets of different sizes and has been tested using sets of thousands of instances.

A major goal in our coding was to ensure scalability. Typically, data mining such as this occurs on a distributed filesystem and uses a map-reduce architecture. This is because the datasets in these scenarios are many gigabytes in size (such as every customer transaction at every Walmart in a year, which would be a staggering number). We noticed that our target dataset (ds5.txt) was small enough to fit in main memory with no major issues, but we still wanted to implement a more general solution. Many parts of our implementation were designed to scale easily, and we made sure to utilize space efficient data structures throughout.

We were tasked with parallelizing the Apriori algorithm, and during our research we discovered the SON (Savasere, Omiecinski, and Navathe) algorithm. This itemset mining algorithm can use several different algorithms as its base (such as PCY or multihash). We decided that the best approach would be to implement this algorithm, using Apriori as the basic itemset finder.

1.2 Significance

When mining for association rules, and in data mining in general, a major metric of efficiency and performance is *passes*. This represents the number of passes that must be made over the dataset to derive intended information from it. In a serial computing environment, only a single processor is available to make these passes over the entire dataset. When mining massive datasets (where the dataset is too large to fit into main memory, or RAM) this can be computationally expensive. File IO operations are expensive, and so any algorithm which can reduce or distribute them offers benefits. This makes data mining a perfect candidate for parallelisation.

1.3 Parallelization and Apriori

In particular, the Apriori algorithm for association rule mining has a lot to gain from parallelization. In Apriori, $k+1$ passes (where k is the size of the largest frequent itemset present) must be made over the dataset before rule generation can take place. If done in serial, this can easily result in trillions of file IO operations being performed by a single processor. Taking advantage of data parallelism and implementing Apriori on a high-performance compute cluster can allow for these expensive IO operations to be evenly distributed among many processors, leading to massive improvements in efficiency.

2. System Design

2.1 Architecture Overview

For this project, we used **C** as we were more familiar with the language. We used **MPI** to distribute and merge the data when working with multiple processes. **UofT Teach** was used as a high-performance computer cluster that gave access to a parallel computing environment, allowing us to perform simulations of the algorithm.

2.2 Scripts Involved

Apriori.c/ Apriori.H

Handles the Apriori algorithm, uses triangular matrix

Dynamic_hash_table.c / Dynamic_hash_table.h

A data structure used to store frequent items and their support

find-rules .c

Our main function. Split into portions, send to processors, gets pairs

Io-processing.c

Processes the input data set,

Rules-aux.c/rules-aux.h

Helper functions for rule generation and itemset generation

2.3 Data Structures

- We used a **dynamic hash table** to keep track of frequently occurring terms. This allows for the size to grow as more are added and uses a hash function to index. There were two benefits to using a hashtable. It allowed us to find unique integers much faster, as duplicate entries are not permitted. It also gave us $O(1)$ lookup for frequent item support, which was useful during pair generation.
- We also used a **triangle matrix** for counting item pairs. Using a triangular matrix allows for less memory usage than an array of triples (num1, num2, count). Due to the way Apriori disqualifies infrequent supersets, there are often far more pairs than any other size of itemset, so storing them space efficiently was important.

- We created another hashtable struct, based on our earlier `dynamic_hashtable`, to store frequent itemsets and their support. The insert, lookup and hash functions in this data structure were designed to be order agnostic. This meant that when called with an itemset, it would return the support of the itemset regardless of the order the items were stored in when they were inserted. This was important as it allowed us to poll support for itemsets generated algorithmically when itemset mining and when creating rules. When creating rules, it is known that the antecedent (lhs) of a rule must be a frequent itemset, meaning we have calculated its support and stored it already. So, the confidence a rule can be found by simply polling the hashtable for the antecedent and itemset itself.

3. Implementation

3.1 Handling

In this project, we used a script called **io-processing.c** to process the input and output. This program splits the input data into **p** processes without overlap, starting each one on a new line in the data, and keeps track of where the chunks start. The data is tokenized and unique occurrences in each chunk are counted using a local hash table, and the decided frequent terms from each chunk are then merged into a global hash table.

We detected frequent items with a commonly used metric called support. An item's support value is calculated by finding the percentage of transactions where it occurs in the dataset. If an item's support is higher than an arbitrary threshold (provided as a command line argument), it is deemed frequent. We had to make slight changes to this formula, as each process sought to determine which items were frequent to its chunk of the dataset, not the dataset as whole. Each process calculated a local support value, based on the percentage of the file it was responsible for processing. In this way, each process creates a hashmap containing all the items that are frequent to its section of the file. We know that this identifies all frequent items because to be frequent in a union of datasets, a value must be frequent in at least one of the composite datasets. Therefore, if we identify all the frequent items in each chunk of our file, we can know for certain we have found all the frequent items in the file as a whole

3.2 Algorithm

1. Set Up

The program starts by handling the input file. If the file is text based (indicated by the user) it is converted to a numeric format, and a hashmap is created to recall the item names when needed. The data is split up based on file size and a hash table with frequent terms is created.

We now implement the SON (Savasere, Omiecinski, and Navathe) algorithm. This algorithm parallelizes Apriori.

2. Distributing Data

Using process-to-process communication, the master process (rank=0) decides where each chunk should start and divides the file into equal sized parts. These split points are then broadcast to their respective process using MPI_Bcast() so they can start working on their respective chunk. Each process then uses a local hash table to count the occurrence of each item, filtering out the infrequent terms.

Merging

MPI_Reduce() is used to combine the transaction count from each process, storing the total number of transactions in the master process. The MPI_Barrier() function is activated to check if all processes are finished with their current work before proceeding.

Following the steps of the SON algorithm, each process now performs the Apriori algorithm on its assigned chunk of the input file.

3. Pair Mining

Each process rereads its chunk, grabbing the frequent items from each transaction. If a transaction contains more than two frequent items, the item pair count is updated in a triangular matrix that keeps track of these item pairs. This matrix is then pruned to filter out the infrequent pairs which fail to meet the support threshold. Due to the frequency of pairs compared to other size of itemset, we handled them as a special case, outside of the general solution to finding itemsets of $k+1$ size. All candidate pairs (from the frequent items) are stored in a triangular matrix for space efficiency. This matrix is then pruned to yield frequent pairs.

4. Size k itemset generation

Using the frequent pairs as a base, each process finds itemsets of increasing size (k), until it finds a size at which no itemsets are frequent. Each time a set of itemsets

is generated, it is pruned based on the support value so that only frequent itemsets are used as the basis for the next size. In this way, the Apriori algorithm prunes low support supersets without having to process them.

5. Rule generation

The master process receives the list of all frequent itemsets from each process and stores them locally. It then generates association rules based on each itemset, only keeping those rules which meet a confidence threshold (supplied by the user as a command line argument)

6. Results

The results are then printed, and the program finishes with `MPI_Finalize()`. The top association rules are printed to stdout, while the list of all confident rules is written to `rules.txt`

3.3 Execution Steps

If the intended input file is in text format, it must be preprocessed by our file `map_items` (which is simply compiled by using `gcc` on `map_items.c`). Our main code can be compiled using the terminal command `"mpicc find-rules.c io-processing.c Apriori.c dynamic_hash_table.c itemset_hash_table.c rules-aux.c -o find-rules"`, and can then be run by using the terminal command `"mpirun -np <p> find-rules <inputfile> <support_threshold> <confidence_threshold> <text>"`, where both thresholds are decimal values between 0 and 1, and `text` is either 0 or 1, to indicate if the input file is in text format (1 for yes).

4. Results

4.1 Output

The output for this program consistently identifies frequent item sets from the given dataset. An example of the output for a trial with 5 processors on `ds5.txt` with a support of 5% for frequent items and a 75% confidence for association rules would be:

```
#####
```

```
Generated 27 association rules with confidence >= 0.750000
```

```
#####
```

Top 10 association rules (numeric):

Rule 1: {36, 39} => {38} (Confidence: 95.48%, Support: 2.21%)

Rule 2: {36, 48} => {38} (Confidence: 96.05%, Support: 1.54%)

Rule 3: {36, 48} => {39} (Confidence: 78.81%, Support: 1.27%)

Rule 4: {38, 41} => {39} (Confidence: 78.29%, Support: 3.46%)

Rule 5: {38, 48} => {39} (Confidence: 76.81%, Support: 6.92%)

Rule 6: {39, 110} => {38} (Confidence: 98.92%, Support: 1.97%)

Rule 7: {39, 170} => {38} (Confidence: 98.06%, Support: 2.29%)

Rule 8: {41, 170} => {38} (Confidence: 98.53%, Support: 1.01%)

Rule 9: {48, 110} => {38} (Confidence: 98.62%, Support: 1.54%)

Rule 10: {48, 170} => {38} (Confidence: 98.78%, Support: 1.74%)

#####

Performance Metrics:

Total execution time: 1.225 seconds

File splitting time: 0.001 seconds (0.11%)

Frequent items mining time: 0.178 seconds (14.56%)

Triangle matrix build time: 0.000 seconds (0.02%)

Pair generation time: 0.155 seconds (12.62%)

Triangle pruning time: 0.000 seconds (0.00%)

Large itemset mining time: 0.463 seconds (37.81%)

Rule generation time: 0.000 seconds (0.00%)

Dataset statistics:

Total transactions: 1057944

Total frequent items: 353

```
Total frequent pairs: 62
Total frequent itemsets: 95
Total association rules: 27
#####
```

4.2 Performance Insights

Execution Time Analysis:

Our results show that most of the execution time was spent on large itemset mining rather than file processing or frequent pair detection. The Apriori-based approach efficiently handled smaller itemsets but became computationally expensive as k increased.

Our implementation reaches optimal performance at 5 processors. We believe that it did not scale well into high values of processors due to the way local support is handled. When more processors are added, but file size is not increased, each processor has a smaller and smaller value for their local support. We had assumed that this would be offset by the fact that each processor is now working with fewer lines, but we found the opposite case true. Processors were especially finding more and more frequent itemsets larger than 2 and spending more time processing and generating these larger itemsets. The smaller values of local support meant that more and more k -size itemsets were seen as valid and retained by local processors, until being pruned by the master processor.

Memory Optimization:

Using a triangular matrix for frequent pairs reduced memory usage, and the dynamic hash table improved lookup efficiency. Further optimizations, such as permanent filtering of infrequent items, could enhance performance.

5. Conclusion

5.1 What Worked Well

Our implementation gave the correct outputs and demonstrated fantastic performance metrics using these measures:

- partitioning the data across processors in an efficient way

- Process-to-process communication
- Scalability of the algorithm (tested using different data sizes)
- Final testing

5.2 Challenges and Future Improvements

Our test case was small enough (and resources on cluster large enough) that we could have stored the entire dataset in main memory, this would have greatly increased efficiency. However, by not doing this, our code remains scalable to datasets which are larger than what we tested on.

A potential improvement that we never implemented was permanent filtering of infrequent items. When each process passes over the file to find pairs of frequent items, it could have written to a new output file, writing only the frequent items from the input file. This file could have then been used for future processing when generating k size itemsets.

We initially had some non-deterministic behavior inside of our pair storage and checking logic, this took some time to iron out. It turned out to be an issue with the way we were recalling items from the triangular matrix. Mapping items to dense indices and then retrieving them from a matrix like this is opaque, and we did not immediately see that our index function was overly complicated and returned negative numbers at some points.

6. Next Steps

For our next step in this project, we would like to see how the performance of our data mining algorithm from a parallel approach compares to a similar sequential data mining algorithm. Despite knowing that the parallel algorithm is more efficient, it would still be interesting to know how much the extra processors speed up the process.

We could also try using algorithms other than Apriori for data mining, such as the sampling algorithm or a frequent-pattern tree algorithm. We could then compare the results, speed, and memory usage of the different programs.

7. References

J. Leskovec, A. Rajaraman, and J. D. Ullman, Mining of Massive Datasets, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2020.

R Elmasri and S.B. Navathe, "Fundamentals of Database Systems," 7th ed. Boston, MA, USA: Pearson, 2015, pp. 1073-1084

T. Xiao, C. Yuan and Y. Huang, "PSO: A Parallelized SON Algorithm with MapReduce for Mining Frequent Sets," *2011 Fourth International Symposium on Parallel Architectures, Algorithms and Programming*, Tianjin, China, 2011, pp. 252-257, doi: 10.1109/PAAP.2011.38.